# Rebalancing & Hot Partitions

*consistent hashing and key salting*

Watch the 3-min video

**In one line:** rebalance with many fixed partitions, dynamic splitting or consistent hashing so only a small fraction of keys move (never hash mod N); a single hot key is a different problem, fixed by salting it across sub-keys and merging on read.

## 1 The celebrity problem

A celebrity with 50 million followers posts a photo. Within seconds, millions of likes hit the **same post ID**, which lives on **one partition**. That shard is pinned at 100% CPU while the other 19 sit idle. Adding nodes will not help: the key still maps to one partition.

Two separate problems hide here. **Rebalancing** moves data between nodes as the cluster changes. **Hot partitions** happen when one key gets far more traffic than any node can serve. We take them in order.

## 2 Rebalancing, and why hash mod N fails

Data has to move between nodes over time: you add nodes as data grows, replace failed ones, or even out load. Good rebalancing **spreads data and load evenly**, keeps the database **available during the move**, and **moves as little data as possible**.

The naive approach is `hash(key) % N`, where N is the number of nodes. It looks simple until N changes. Going from 4 nodes to 5 changes the result for about **80% of keys**, so almost all your data must move at once.

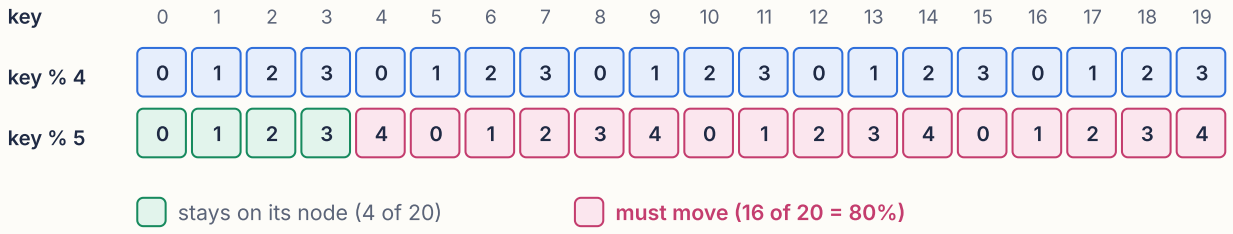



**Figure 1.** With key % 4 vs key % 5, only keys 0–3 (and every 20th key after) keep the same node. In general, going from N to N+1 nodes moves about N/(N+1) of the keys.

## 3 Three ways to move only a little

**1. Many fixed partitions.** Create far more partitions than nodes up front, say 1,000, and give each node several. A key **always maps to the same partition**; only the partition-to-node assignment changes. When a node joins, it takes whole partitions from the others, and nothing else moves. Redis Cluster uses **16,384 hash slots** this way; Elasticsearch has a fixed number of primary shards per index.

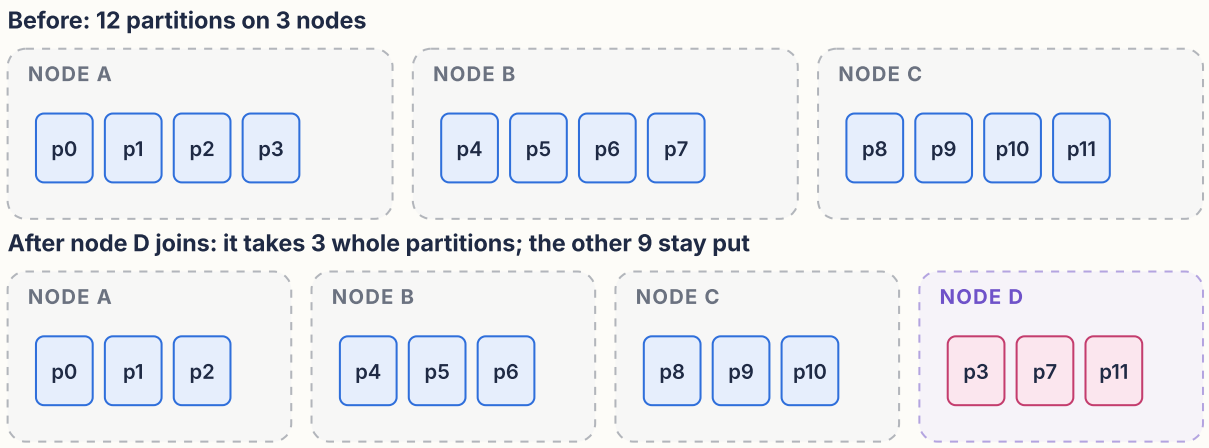



**Figure 2.** Keys never change partition. Node D steals one partition from each existing node, so 3 of 12 partitions (25%) move, the minimum for an even spread over 4 nodes.

**2. Dynamic splitting.** When a partition grows too big, it splits in two, and one half can move to another node. HBase regions, MongoDB chunks and DynamoDB work this way. The number of partitions grows with the data.

**3. Consistent hashing with virtual nodes.** Nodes and keys are placed on a hash ring; each key belongs to the next node clockwise. With many virtual nodes per server (Cassandra), a new node takes **small slices from many nodes**, so only about **1/N of the keys move**, spread across the cluster instead of hitting one neighbour.

| Approach | When a node joins | Used by |
|---|---|---|
| **hash % N** | ~80% of keys move (4 → 5 nodes) | avoid for stored data |
| **Fixed partitions** | whole partitions move; keys keep their partition | Redis Cluster (16,384 slots), Elasticsearch |
| **Dynamic splitting** | big partitions split; halves move | HBase, MongoDB, DynamoDB |
| **Consistent hashing + vnodes** | ~1/N of keys move, from many nodes | Cassandra |

## 4 Hot keys: salting

Rebalancing cannot fix a **single hot key**, because one key always maps to one partition. The fix is **salting**: append a random suffix, 0 to 9, to the hot post ID. Writes now spread across **10 sub-keys**, which hash to different partitions.

The cost lands on reads: they must fetch all 10 sub-keys and combine them, for example by **summing the like counts**. So salt only **known hot keys**, not every key.

The code:

```
SALTS = 10

def record_like(post_id):
    salt = random.randint(0, SALTS - 1)
    db.incr(f"likes:{post_id}#{salt}")      # one write, one random bucket

def total_likes(post_id):
    keys = [f"likes:{post_id}#{s}" for s in range(SALTS)]
    return sum(db.mget(keys))               # ten reads, one sum
```

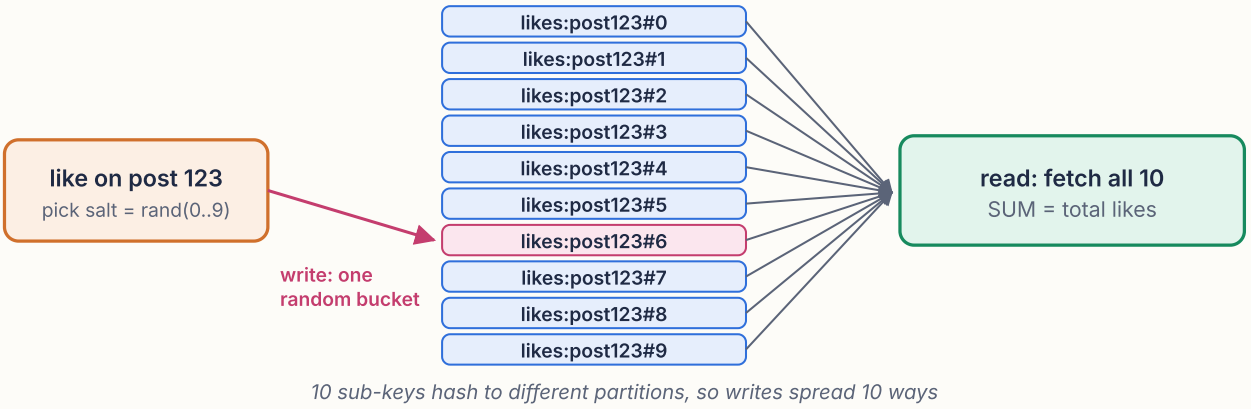



**Figure 3.** Each like goes to one random bucket; the total is the sum of all ten buckets.

Pair this with **caching** for hot reads (the total can be a few seconds stale), and **detect hot keys automatically** from per-key traffic metrics, so you can salt them or isolate them on a dedicated partition before they hurt.

**WORKED EXAMPLE: THE CELEBRITY POST**

- Suppose likes peak at **200,000 writes/s** on post 123, and one partition can absorb about **25,000 writes/s**. Unsalted: the partition is 8× overloaded, whatever the cluster size.
- Salt into **10 buckets**: about 20,000 writes/s per bucket, each on a different partition. Every bucket is under its limit.
- Reads now cost 10 key fetches. Cache the summed count for 2 seconds, and 1 million page views per second become one 10-key read every 2 seconds.

**COMMON MISTAKES**

- Placing data with `hash % N`, then discovering that adding one node moves nearly everything.
- Adding nodes to fix a hot key: the key still lives on one partition.
- Salting every key, which makes every read a 10-way fan-out.
- Too few fixed partitions (e.g. 8): you can never grow past 8 nodes without re-partitioning.

## 5 Key terms

**Rebalancing**: moving partitions between nodes

**Hash slot**: Redis Cluster's fixed partition (16,384)

**Virtual node**: many ring positions per server

**hash % N**: naive placement; breaks when N changes

**Dynamic splitting**: split a partition when it grows too big

**Hot key**: one key with far more traffic than others

**Fixed partitions**: many more partitions than nodes; move whole

**Consistent hashing**: hash ring; ~1/N of keys move

**Salting**: random suffix spreads a hot key

## 6 Interview questions

**Q1. How do you rebalance shards with minimal data movement?** BASIC

Avoid hash % N. Use many fixed partitions that move whole between nodes, dynamic splitting, or consistent hashing with virtual nodes, so only a small fraction of keys (about 1/N) moves.

**Q2. Why is hash % N bad for rebalancing?** BASIC

Changing N changes almost every key's result. From 4 to 5 nodes about 80% of keys move; from 10 to 11, about 91%. That means a cluster-wide data migration for one new node.

**Q3. How does Redis Cluster place keys?** BASIC

CRC16(key) mod 16,384 picks a hash slot; slots are assigned to nodes. Adding a node moves whole slots to it; keys never change slot.

**Q4. One shard sits at 100% CPU while 19 are idle. Diagnose and fix.** SCENARIO

Check per-key metrics. If one key is hot, adding nodes won't help: salt it across e.g. 10 sub-keys and merge on read, cache its reads, or isolate it on a dedicated partition. If the partition is just large or hot overall, split it and move half.

**Q5. You salt a like counter into 10 buckets. What does a read cost?** SCENARIO

Ten key reads and a sum, instead of one read. Batch them in one multi-get and cache the total for a second or two, so the read side stays cheap.

**Q6. How many fixed partitions would you create for a cluster of 10 nodes today?** SCENARIO

Many more than nodes, e.g. 1,000 (100 per node). That allows growth to hundreds of nodes without re-partitioning, while keeping per-partition overhead small.

**Q7. Does consistent hashing alone fix hot spots?** TRICKY

No. It limits how many keys move when nodes change. Without virtual nodes, load can still be uneven; and no placement scheme spreads one hot key, which always maps to one node.

**Q8. Why not salt every key to be safe?** TRICKY

Every read becomes a fan-out to all buckets: 10× the read work and a merge step, plus tail latency from the slowest bucket. Salt only keys you detect as hot.

Next: SD-E-025 · Secondary Indexes in Partitioned Data backendroadmap.com